

Contents

1	DD10: 監査ログ書込・完全性検証	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	2
1.4.1	3.1 チェーンセグメント分割方針 (長大化対策)	2
1.4.2	3.2 ハッシュチェーン	2
1.4.3	3.3 retention_class 自動付与	3
1.4.4	3.4 IP マスク	3
1.4.5	3.5 9 カテゴリ action コード網羅表	4
1.4.6	3.6 監査ログ完全性検証	4
1.4.7	3.7 tombstone 本文削除と hash 保持の整合性検証	5
1.4.8	3.8 retention_class 3 区分	6
1.5	4. 関連設計	6
1.6	5. テスト観点	7
1.6.1	5.1 その他観点	7
1.7	6. 未確定事項・確認事項	7

1 DD10: 監査ログ書込・完全性検証

1.1 0. 文書情報

項目	内容
文書名	DD10: 監査ログ書込・完全性検証
詳細設計 ID	DD10
対象システム	FAQ AI ウィジェット SaaS / メインシステム
関連機能 ID	NFR-601~606 (監査ログ網羅 / 完全性検証) / AC-026 / AC-031 / AC-032
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
機能	NFR-601	監査ログ網羅 9 カテゴリ
機能	NFR-602	監査ログ完全性検証
機能	NFR-602c	tombstone 方式
ライブラリ	app/shared/src/lib/audit-hash.ts app/shared/src/lib/audit.ts	ハッシュチェーン + 書込
cron	audit-chain-verify.ts audit-tombstone-consistency.ts tombstone-batch.ts	完全性検証 + tombstone 化
テーブル	audit_logs	監査ログ基盤

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 10.7.0	チェーンセグメント分割方針	月次セグメント YYYY-MM
§ 10.7.1	ハッシュチェーン	SHA-256 + prev_hash 連結
§ 10.7.2	retention_class 自動付与	action コード prefix マップ
§ 10.7.3	IP マスク	default /24 + /48 / gdpr_enhanced /16 + /32
§ 10.7.4	9 カテゴリ action コード	§ 15.2.1 網羅表参照
§ 10.8	監査ログ完全性検証	日次全件再計算 + R2 ダイジェスト
§ 10.8.1	tombstone 本文削除と hash 保持の整合性検証	月次バッチ

元章	元タイトル	概要
§ 15.2	監査ログ	9 カテゴリ action コード網羅表 / retention_class / tombstone

1.4 3. 詳細設計本文

1.4.1 3.1 チェーンセグメント分割方針 (長大化対策)

ハッシュチェーンは「契約単位 × 月次セグメント」を原則とする。`segment_key = YYYY-MM` をキーとして、チェーンの `prev_hash` 連結は同セグメント内のみで完結させる。月初の最初の行は前月末の最後の行の `current_hash` を `prev_segment_hash` として別カラムに記録し、セグメント間の連続性を担保する。

これにより、運営者操作 (`contract_owner_user_id=NULL`) のグローバルチェーンも月次セグメントに分割され、日次検証バッチの $O(N)$ コストを 30 日分のみに抑制する。差分検証 (前日分の `prev_hash` 連結のみ再計算) との併用で、性能目標は次の通り。

項目	目標値
日次差分検証	前日分 ☑ 5 分
月次フル検証 (1 セグメント)	☑ 30 分
年次総合検証	☑ 8 時間

`audit_logs` テーブルに `segment_key TEXT NOT NULL` カラムと `prev_segment_hash TEXT` カラムを追加する DDL マイグレーションは [../02_基本設計/03_テーブル設計.md](#) / [../04_運用設計/index.md](#) で別途定義する。

1.4.2 3.2 ハッシュチェーン

```
// app/shared/src/lib/audit-hash.ts
export async function computeChainHash(prevHash: string | null, fields: Record<string, unknown>) {
  const canonical = JSON.stringify(Object.keys(fields).sort().reduce((acc, k) => {
    acc[k] = fields[k]; return acc;
  }, {} as Record<string, unknown>));
  const input = `${prevHash ?? ''}|${canonical}`;
  const enc = new TextEncoder().encode(input);
  const buf = await crypto.subtle.digest('SHA-256', enc);
  return base64urlEncode(new Uint8Array(buf));
}
```

```
// app/shared/src/lib/audit.ts
export async function writeAudit(env: Env, input: AuditInput) {
  // 直前の current_hash を取得 (契約単位チェーン)
  const prev = await env.DB.prepare(
    `SELECT current_hash FROM audit_logs WHERE contract_owner_user_id = ?1 ORDER BY created_at`
  ).bind(input.contractOwnerUserId ?? null).first<{ current_hash: string } | null>();
  const fields = {
    id: input.id ?? generateUlid(),
    contractOwnerUserId: input.contractOwnerUserId ?? null,
    actorUserId: input.actorUserId ?? null,
    actorRole: input.actorRole ?? null,
    action: input.action,
    targetType: input.targetType ?? null,
    targetId: input.targetId ?? null,
    ipAddressMasked: input.ip ? maskIp(input.ip) : null,
    metadata: input.metadata ?? null,
    retentionClass: input.retentionClass,
    createdAt: new Date().toISOString(),
  };
  const currentHash = await computeChainHash(prev?.current_hash ?? null, fields);
  await env.DB.prepare(`
    INSERT INTO audit_logs (id, contract_owner_user_id, actor_user_id, actor_role, action, targ
      ip_address_masked, metadata, retention_class, prev_hash, current_has
    VALUES (?1, ?2, ?3, ?4, ?5, ?6, ?7, ?8, ?9, ?10, ?11, ?12, ?13)
  `).bind(fields.id, fields.contractOwnerUserId, fields.actorUserId, fields.actorRole,
```

```

        fields.action, fields.targetType, fields.targetId, fields.ipAddressMasked,
        fields.metadata, fields.retentionClass, prev?.current_hash ?? null, currentHash,
        fields.createdAt).run();
    }
}

```

1.4.3 3.3 retention_class 自動付与

```

const RETENTION_BY_ACTION_PREFIX: Record<string, RetentionClass> = {
  'auth.': 'general',
  'owner.': 'general',
  'project.': 'general',
  'faq.': 'general',
  'inquiry.': 'general',
  'chat.': 'general',
  'billing.': 'billing',           // 7 年
  'usage.': 'billing',
  'data.restore.': 'operator_high_priv',
  'owner.suspend': 'operator_high_priv',
  'owner.resume': 'operator_high_priv',
  'threshold.update': 'operator_high_priv',
  'rate_limit.override': 'operator_high_priv',
};

// CI 検証: action コードを書き込む全箇所ですべて map にヒットすることを保証
// 未登録の action コードは write 時に throw (`UNCATEGORIZED_ACTION_CODE`)

```

1.4.4 3.4 IP マスク

```

// app/shared/src/lib/ip-mask.ts
export type IpMaskMode = 'default' | 'gdpr_enhanced';

export function maskIp(ip: string, mode: IpMaskMode = 'default'): string {
  if (ip.includes(':')) {
    if (mode === 'gdpr_enhanced') {
      // IPv6: 末尾 96 ビット (6 グループ) を 0 化 — /32 相当まで縮約
      const parts = ip.split(':');
      return parts.slice(0, 2).join(':') + ':0:0:0:0:0:0';
    }
    // IPv6: 末尾 80 ビット (5 グループ) を 0 化 — /48 相当
    const parts = ip.split(':');
    return parts.slice(0, 3).join(':') + ':0:0:0:0:0';
  }
  if (mode === 'gdpr_enhanced') {
    // IPv4: 末尾 2 オクテット (/16 相当) を 0 化 — 個人識別性をさらに低減
    const parts = ip.split('.');
    return parts.slice(0, 2).concat(['0', '0']).join('.');
  }
  // IPv4: 末尾 1 オクテット (/24 相当) を 0 化 — 既定値
  const parts = ip.split('.');
  return parts.slice(0, 3).concat(['0']).join('.');
}

```

1.4.4.1 3.4.1 GDPR 適用利用者向け強化マスクオプション GDPR (EU 一般データ保護規則) の Recital 30 / Article 4(1) では IP アドレスを **オンライン識別子** と位置づけ、Article 5 / 25 (by design) で「目的に必要な最小限の個人データ」を要求している。IPv4 末尾 1 オクテット削除 (/24) は実務上の「IP 個人識別性低減」として広く採用されるが、EU 監督機関 (CNIL, ICO 等) の事例では /24 でも個人識別可能性を否定しないケースがあり、GDPR 適用契約では追加の縮約を提供する。

項目	既定 (default)	GDPR 強化 (gdpr_enhanced)
IPv4	末尾 1 オクテット 0 化 (/24 相当、例 203.0.113.0)	末尾 2 オクテット 0 化 (/16 相当、例 203.0.0.0)
IPv6	末尾 80 ビット 0 化 (/48 相当)	末尾 96 ビット 0 化 (/32 相当)
適用判定	全オーナー (デフォルト)	contract_owners.gdpr_applicable=true のオーナー配下のみ
適用箇所	audit_logs.ip_masked / error_logs.ip_masked / Cloudflare Log-push 二次加工 / question_logs 等の IP 記録列すべて	
副作用	レート制限 / 不正検知の精度低下 - /16 までマスクすると同一 ISP / 同一国内事業者から大量アクセスを区別できない	

運用ガイド:

- **IP マスクモード:** default 固定 (/24 + /48)。
- **不正検知への影響緩和:** GDPR 強化モード時はレート制限の判定キーを ip_masked → user_id (またはセッション ID) に切替 (不正検知バッチで ip_mask_mode='gdpr_enhanced' 契約には user_id キーを優先)。
- **監査ログ整合性:** マスク方式の変更は適用日時を contract_owners.ip_mask_mode_changed_at に保存、変更前後の audit_logs.ip_masked 値は混在する想定で検証バッチを設計 (hash 連鎖検証は影響なし、列値の比較分析時にモード境界を考慮)。

1.4.5 3.5 9 カテゴリ action コード網羅表

カテゴリ	action コード例	retention_class
認証関連	auth.signup, auth.login_success, auth.login_failure, auth.logout, auth.password_reset, auth.re_auth.success, auth.ip_blocked, auth.terms_agree	general
認可境界	authz.owner_boundary_violation, authz.project_boundary_violation	general
課金・請求	billing.invoice.issue, billing.invoice.paid, billing.budget_limit_change, billing.payment_failed, usage.warning.80, usage.limit.100, usage.limit.125	billing
連携 IF	integration.if1.receive, integration.if6.receive, integration.if9.send, integration.if10.event.invoice.paid	general
データエクスポート・復元	data.export.request, data.export.complete, data.restore.execute	operator_high_priv (restore のみ) / general (他)
一括操作	faq.bulk_import, inbox.read_all	general
論理削除 / 物理削除 (v1.10)	user.logical_delete, project.logical_delete, project_users.logical_delete, user.hard_delete_by_batch, project.hard_delete_by_batch, project.created_with_owner_admin (プロジェクト作成時のオーナー admin 行自動 INSERT)	general
セキュリティ運用	security.rate_limit.override, security.threshold.update, security.cache.invalidate, security.session.force_invalidate	operator_high_priv
DB 直接クエリ	db.direct_query.execute	operator_high_priv

1.4.6 3.6 監査ログ完全性検証

```
// app/workers/cron/src/jobs/audit-chain-verify.ts (JST 03:00)
export async function verifyAuditChain(env: Env) {
  const contractOwnerUserIds = await env.DB.prepare(
    `SELECT DISTINCT COALESCE(contract_owner_user_id, '__operator__') as tid FROM audit_logs`
  ).all<{ tid: string }>();

  for (const t of contractOwnerUserIds.results) {
    const rows = await env.DB.prepare(
      `SELECT * FROM audit_logs
      WHERE COALESCE(contract_owner_user_id, '__operator__') = ?1 AND tombstone = 0
      ORDER BY created_at`
    ).all<{ }>();
  }
}
```

```

`.bind(t.tid === '__operator__' ? null : t.tid).all();

let prevHash: string | null = null;
for (const row of rows.results) {
  const fields = extractAuditFields(row);
  const expected = await computeChainHash(prevHash, fields);
  if (expected !== row.current_hash) {
    await enqueueServiceAlert(env, {
      level: 'critical',
      message: `監査ログ完全性違反検出: owner=${t.tid} id=${row.id}`,
    });
    // contract_owners 行存在で「オーナー」を判定するため、IS NOT NULL チェックで対象を絞る
    await env.DB.prepare(`UPDATE contract_owners SET audit_dual_chain_enabled = 1 WHERE use
      .bind(t.tid).run();
    break;
  }
  prevHash = row.current_hash as string;
}
}

// R2 へ日次の全件再計算結果ダイジェストを署名付きで保管
const digest = await computeFullChainDigest(env);
await env.R2_AUDIT.put(
  `audit/digests/${new Date().toISOString().slice(0,10)}.json`,
  JSON.stringify({ digest, signature: await sign(env, digest) }),
);
}

```

1.4.7 3.7 tombstone 本文削除と hash 保持の整合性検証

通常の verifyAuditChain は tombstone=0 (現役データ) のみ対象とするが、tombstone=1 (本文削除済) の行も hash 連鎖の一部 なので、別途の整合性検証が必要。

1.4.7.1 3.7.1 tombstone 行の構造

tombstone=0 の行: action, actor_id, target_id, before/after, current_hash すべて存在
 tombstone=1 の行: action, actor_id, current_hash のみ残し、PII を含むカラム (before/after, ip, meta

1.4.7.2 3.7.2 整合性検証バッチ (AuditTombstoneConsistencyVerifier) AuditChainVerifier-Worker (JST 03:00) と並行して、月次 (毎月 1 日 JST 04:00) に以下を実行:

1. tombstone=1 行の hash 整合性: 隣接する tombstone=0 行との prev_hash 連鎖が壊れていないか検証
2. R2 アーカイブとの突合: tombstone=1 化された時点で R2 audit-archive/<retention_class>/<year>/<month>.tar.gz に元データが退避されているか (R2 オブジェクト存在 + JSONL 内の id 列との一致を verify)
3. R2 アーカイブと audit_logs の id 集合一致: 月次で R2 内の id 集合と D1 audit_logs (tombstone=1 AND created_at が該当月) の id 集合が完全一致することを確認
4. R2 アーカイブの完全性: 月次 R2 オブジェクトを Cloudflare R2 の etag で検証 + 抽出後の hash 計算で hash 列との一致を確認

```

// app/workers/cron/src/jobs/audit-tombstone-consistency.ts (月次 JST 04:00)
export async function verifyTombstoneConsistency(env: Env) {
  const lastMonth = getLastMonthRange(); // YYYY-MM
  for (const retentionClass of ['5y', '7y']) { // 1y は archive 対象外
    // (a) D1 から tombstone=1 行を取得
    const d1Rows = await env.DB.prepare(`
      SELECT id, current_hash FROM audit_logs
      WHERE retention_class = ?1 AND tombstone = 1
      AND substr(created_at, 1, 7) = ?2
    `).bind(retentionClass, lastMonth).all<{ id: string; current_hash: string }>();

    // (b) R2 アーカイブから対応データ取得

```

```

const r2object = await env.R2_AUDIT.get(`audit-archive/${retentionClass}/${lastMonth}.tar.g
if (!r2object) {
  await enqueueServiceAlert(env, {
    level: 'critical',
    message: `R2 アーカイブ欠落: ${retentionClass}/${lastMonth}`,
  });
  continue;
}
const r2Rows = await unpackArchive(r2object);

// (c) id 集合の完全一致
const d1Ids = new Set(d1Rows.results.map(r => r.id));
const r2Ids = new Set(r2Rows.map(r => r.id));
const missing = [...d1Ids].filter(id => !r2Ids.has(id));
const extra = [...r2Ids].filter(id => !d1Ids.has(id));
if (missing.length > 0 || extra.length > 0) {
  await enqueueServiceAlert(env, {
    level: 'critical',
    message: `tombstone/archive 不整合: missing=${missing.length}, extra=${extra.length}, mo
  });
}

// (d) hash 連鎖検証 (tombstone=1 を含めて全行で連鎖)
// (実装は §10.7.1 と同じロジック、tombstone=0/1 を区別せず連鎖計算)
}

await writeAudit(env, {
  action: 'audit.chain.tombstone.verify', retentionClass: '5y',
});
}

```

1.4.7.3 3.7.3 アラート時の対応

- **R2 オブジェクト欠落**: critical → 復元手順 (RB-016) を即時実行、原因究明
- **id 集合不一致**: critical → 該当行を個別調査、tombstone バッチ と R2AuditArchive のジョブ実行記録を確認
- **hash 連鎖断絶**: critical → tombstone=1 行が攻撃で改ざんされた可能性、セキュリティインシデント（完全性検証失敗）発動 → 4-eyes 緊急対応 + 法的相談

1.4.7.4 3.7.4 テスト `it-audit-tombstone-consistency-001` で:

- 正常ケース: tombstone=1 行と R2 アーカイブの id 集合が一致、hash 連鎖が継続
- 異常ケース 1: R2 オブジェクトを意図的に削除 → critical 発火
- 異常ケース 2: D1 の tombstone=1 行を意図的に DELETE → id 集合不一致検知

1.4.8 3.8 retention_class 3 区分

`general` 1年 / `operator_high_priv` 5年 / `billing` 7年。tombstone フラグで本文を物理削除しつつチェーン継続性を維持。tombstone 化のバッチ実装は [DD14_ バッチ・非同期処理.md](#) § 14.1.8 を参照。

1.5 4. 関連設計

種別	参照先
要件	../01_ 要件定義/index.md
基本設計	../02_ 基本設計/index.md
セキュリティ設計	../02_ 基本設計/09_ セキュリティ設計.md
テーブル設計	../02_ 基本設計/03_ テーブル設計.md
共有概念	../共有/共有概念.md
運用設計	../04_ 運用設計/index.md

種別	参照先
将来対応	../05_future/index.md
関連 DD	DD09_ 認可ヘルパ.md / DD11_ 暗号化・管理.md / DD14_ バッチ・非同期処理.md

1.6 5. テスト観点

AC ID	テスト ID	テスト方式	テストファイル
AC-026	it-audit-coverage-001	Integration	workers/main-api/test/integration/audit/coverage.test.ts
AC-031	it-audit-chain-001	Daily cron + Manual	workers/cron/test/integration/audit-chain-verify.test.ts
AC-032	it-tombstone-001	Integration	workers/cron/test/integration/tombstone.test.ts

1.6.1 5.1 その他観点

観点	内容
単体	computeChainHash 純関数 / maskIp default / gdpr_enhanced 両モード
結合	writeAudit で 9 カテゴリすべての action コードが retention_class マップにヒット
異常系	未登録 action コード → UNCATEGORIZED_ACTION_CODE throw
改ざん検知	任意の audit_logs 行を直接 UPDATE → 日次 verify で audit_dual_chain_enabled=1 フラグ立て + critical
月次	verifyTombstoneConsistency で R2 オブジェクト欠落 / id 集合不一致 / hash 連鎖断絶を検出
性能	日次差分検証 ☑ 5 分（前日分のみ） / 月次フル検証 ☑ 30 分

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
TBD-DD10-001	月次フル検証の X 万行で Y 分の実測値	中	未確定（SRE）
TBD-DD10-002	年次総合検証の実測値	中	未確定（SRE）
TBD-DD10-003	既存データの segment_key バックフィル手順	中	未確定（開発リード）